2.视觉表现

这节课主要介绍了css中视觉表现相关的属性，如半透明、圆角、2d/3d变换等效果；这一章的涉及的属性非常多，我们主要通过例子来学习；

2.1 opacity

opacity可以控制元素的透明度，属性计算值范围是0～1，初始值是1.0，它有以下特性：

- 没有继承性，父、子元素同时设置半透明时，半透明效果是叠加的
- 在所有支持CSS过渡和动画的CSS属性中，opacity属性是性能最高的
- opacity值不为1的元素会创建一个层叠上下文

2.2 层叠上下文

我们假定用户正面向（浏览器）视窗或网页，而 HTML 元素沿着其相对于用户的一条虚构的 z 轴排开，**层叠上下文**就是对这些 HTML 元素的一个三维构想。

文档中的层叠上下文由满足以下任意一个条件的元素形成：

- 文档根元素（`<html>`）；
- `position` 值为 `absolute`（绝对定位）或 `relative`（相对定位）且 `z-index` 值不为 `auto` 的元素；
- `position` 值为 `fixed`（固定定位）或 `sticky`（粘滞定位）的元素（沾滞定位适配所有移动设备上的浏览器，但老的桌面浏览器不支持）；
- flex（`flexbox` (en-US)）容器的子元素，且 `z-index` 值不为 `auto`；
- grid（`grid`）容器的子元素，且 `z-index` 值不为 `auto`；
- `opacity` 属性值小于 `1` 的元素（参见 the specification for opacity）；
- `mix-blend-mode` 属性值不为 `normal` 的元素；
- 以下任意属性值不为 `none` 的元素：
  - `transform`
  - `filter`
  - `perspective`
  - `clip-path`
  - `mask` / `mask-image` / `mask-border`
- `isolation` 属性值为 `isolate` 的元素；
- `-webkit-overflow-scrolling` 属性值为 `touch` 的元素；
- `will-change` 值设定了任一属性而该属性在 non-initial 值时会创建层叠上下文的元素（参考这篇文章）；

- `contain` 属性值为 `layout`、`paint` 或包含它们其中之一的合成值（比如 `contain: strict`、`contain: content`）的元素。

- 层叠上下文可以包含在其他层叠上下文中，并且一起创建一个层叠上下文的层级。
- 每个层叠上下文都完全独立于它的兄弟元素：当处理层叠时只考虑子元素。
- 每个层叠上下文都是自包含的：当一个元素的内容发生层叠后，该元素将被作为整体在父级层叠上下文中按顺序进行层叠。

在层叠上下文中，我们通常会以z-index来判定元素的层叠关系，但除此之外还有一些隐式的规则：

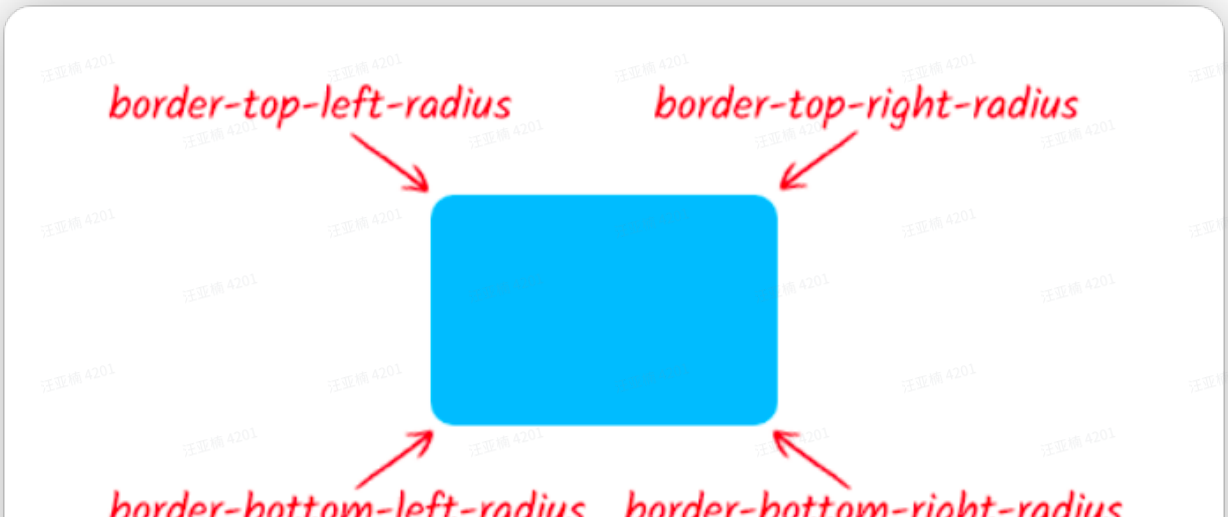
除了z-index外，优先级主要根据内容的重要性来排列，越重要的层级越高；当层级相同的时候，根据文档流，“后来居上”

https://codepen.io/dqqbl/pen/MWrQqKK

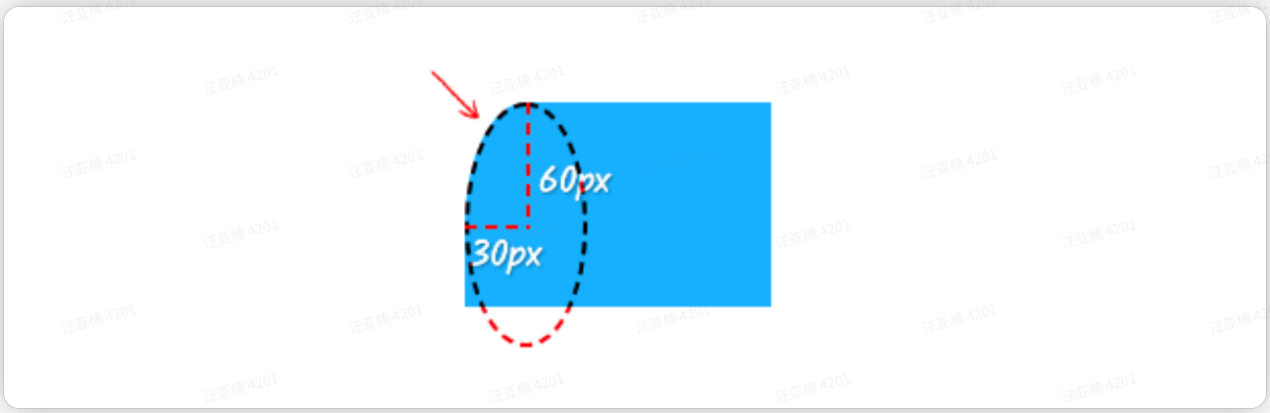
2.3 border-radius

border-radius有以下值得注意的点:

- 百分比取值是相对于元素占据尺寸的百分比，即包含padding和border
- 我们平时书写的 `border-radius: 50%`，`border-radius` 是 border-top-left-radius、border-top-right-radius、 border-bottom-left-radius和border-bottom-right -radius这4个属性的缩写，表示盒模型的四个角



而圆角值 `50%` 则是 `50% 50%` 的缩写，第一个值表示水平半径，第二个值是垂直半径



- 当圆角值过大，出现圆角曲线重叠问题的时候会根据公式计算最终的圆角半径

$$f = \min(L_h/S_h,\ L_v/S_v)$$

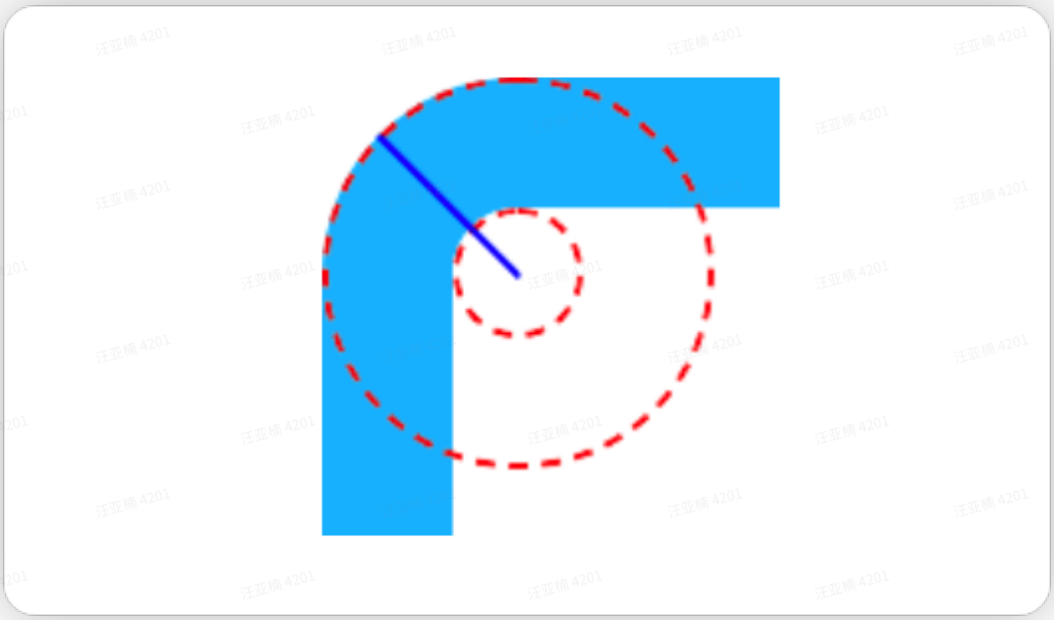
其中S是半径之和，L是元素宽高，下标h和v表示方向，f是计算值。如果计算值f小于1，则所有最终的圆角半径都乘以f；

如下面两个圆角取值结果是相同的，套入公式 `f = min (150/60, 100/200) = 0.5`；

每个圆角值乘0.5可得结果

https://codepen.io/dqqbl/pen/popaxEW

- 如果元素设置了border边框，则圆角半径会被分成内半径和外半径



2.4 box-shadow

box-shadow取值由水平偏移，垂直偏移，模糊半径，扩散半径组成，还有一个关键字inset表示内阴影

CSS

```
/* x偏移量 | y偏移量 | 阴影模糊半径 | 阴影扩散半径 | 阴影颜色 */
box-shadow: 2px 2px 2px 1px rgba(0, 0, 0, 0.2);
```

https://codepen.io/dqqbl/pen/JjMpmOV

使用box-shadow会有一些性能消耗，但是一般页面使用是完全没有问题的，除非页面非常复杂，加了很多渐变滤镜阴影等，对于css性能方面的问题我们后面再专门讲解；

除此之外主要是一些使用技巧

- 通过内阴影模拟边框，使得有边框按钮和无边框按钮的尺寸完全一致

https://codepen.io/dqqbl/pen/wvpyYyJ

- 通过多个box-shadow模拟加载效果

https://codepen.io/dqqbl/pen/mdpXzKP

2.5 2D变换

2d变换主要通过transform属性实现

2.5.1 基本方法

```CSS
/* 位移 */
transform: translate(0, 0);
/* 旋转 */
transform: rotate(0deg);
/* 缩放 */
transform: scale(1);
/* 斜切 */
transform: skew(0deg);
```

需要注意的点：

- translate 位移

  translate(10px)等同于 translate(10px, 0)，也等同于translateX(10px)，**不是** translate(10px, 10px) 的缩写

- rotate 旋转

  rotate的值有四种单位可选：

  - 角度（deg）：角度范围为0～360度，角度为负值可以理解为逆时针旋转。例如，－45deg可以理解为逆时针旋转45度。
  - 百分度（grad）：一个梯度，或者说一个百分度表示1/400个整圆。因此100gads相当于90deg，它和deg单位一样支持负值，负值可以理解为逆时针方向旋转。
  - 弧度（rad）：1弧度等于180/π度，或者大致等于57.3度。1.5708rad相当于100gads或是90deg
  - 圈数（turn）：1圈表示360度，1turn=360deg

  平时我们使用 deg 即可，其余的常用于特殊场景，如运动轨迹等

- scale 缩放
  - 仅支持数值，不支持百分比值
  - 与translate不同，scale(2) 等于 scaleX(2), scaleY(2)
  - 负值可以实现水平翻转
- skew 斜切
  - skew(10deg) 等于 skew(10deg, 0) 等于 skewX(10deg)

https://codepen.io/dqqbl/pen/oNpEQvj

2.5.2 transform的特性

- 不会改变盒模型的实际尺寸和位置，类似于相对定位

- 内联元素（不包括替换元素）不支持transform变换，如果想要对内联元素使用，可将display值设置为`inline-block`
- 当旋转出现锯齿时，是由于显示器精度不足导致，要么使用算法进行边缘模糊计算（虚化），要么换高精度的显示器
- 多个transform值，下一个的计算基于上一个值的计算结果

CSS

```
transform: translateX(40px) scale(.75);
transform: scale(.75) translateX(40px);
```

- 先执行 clip/clip-path 再执行 transform
- 性能表现优秀，对于能使用transform实现的动画效果，优先使用transform
- 会创建层叠上下文
- 使用transform属性的元素会成为后代，固定定位和绝对定位元素的“包含块”
- transform变换本质是调用矩阵函数进行变化

CSS

```
 transform: matrix(a, b, c, d, e, f);
```

2.5.3 transform-origin

transform-origin属性可以用来改变元素变换的中心坐标点，支持xyz轴；除z轴只支持数值，其余轴支持百分比值、数值和关键字属性值（left | center | right | top | bottom）

CSS

```
transform-origin: 2px 30% 10px;
```

transform-origin的使用场景有：

- 模拟现实世界物体的运动

  如蜡烛火焰摆动的效果transform-origin属性值为bottom，钟摆运动的动画效果，需要设置transform-origin属性值为top

- 布局与定位

  我们知道chrome中字体大小最小为12px，想要实现10px大小的字符，可以先设置12px大小的字符，然后缩放一下；但是这样做有一个问题，transform变换的时候元素的原始位置是保留的，这会导致元素缩小后原本设定的间距变大，此时可以使用 transform-origin属性优化一下

https://codepen.io/dqqbl/pen/rNpJQOg

2.6 calc()

calc()函数支持加减乘除4种运算，基本上数值类型的属性都可以使用，需要注意的点：

- 运算符前后带单位或者带百分号的值只能进行加减运算，不能进行乘除运算
- 加号和减号左右两边一定要有空格，否则浏览器无法区分负号和减号，乘法和除法符号两侧无须空格，但是为了格式一致、便于阅读，一般也设置空格

2.7 min()/max()/clamp()

- min()：取最小参数，但实际上这个函数是用来限制最大值的，可以替代max-width
- max()：取最大参数，但实际上这个函数是用来限制最小值的，可以代替min-width

CSS

```
width: 30%;
min-width: 200px;
/* 等同于 */
width: max(200px, 30%);
```

- clamp(MIN, VAL, MAX)：用于区间限制，VAL表示首选值

兼容性方面，除ie外现代浏览器皆支持

2.8 渐变

2.8.1 linear-gradient 线性渐变

线性渐变值由渐变方向🧭和颜色断点列表组成

```
linear-gradient([ <angle> | to <side-or-corner> ,]? <color-stop-list>
)
```

https://codepen.io/dqqbl/pen/rNpJoLM

需要注意的点：

- 默认方向向下
- 单位为角度时，垂直方向是0度，顺时针旋转是正角度
- 没有指定断点位置时，颜色范围均分
- 颜色过渡点默认为两种颜色间的50%，可通过设置不带颜色的断点值改变过渡点

来看看一个颜色断点的例子

https://codepen.io/dqqbl/pen/XWVZoeR

2.8.2 radial-gradient 径向渐变

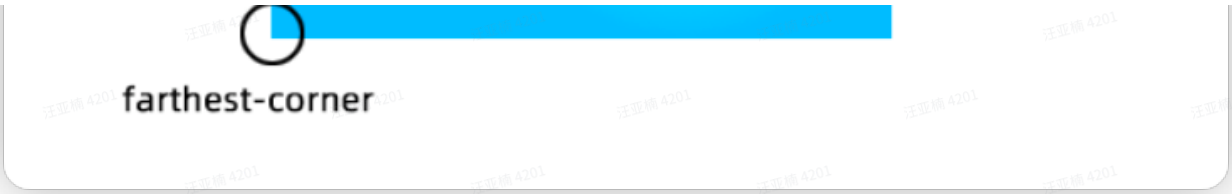
标准语法：

```
CSS
radial-gradient(
  [
    [ circle || <length> ] [ at <position> ]? , |
    [ ellipse || [ <length> | <percentage> ]{2} ] [ at <position>]? , |
    [ [ circle | ellipse ] || <extent-keyword> ] [at <position> ]? ,|
    at <position> ,
  ]?
  <color-stop-list> [ , <color-stop-list> ]+
)
```

可以划分为：渐变形状，渐变终止点，渐变中心点，颜色断点

渐变终止点<extent-keyword>的值

关键字	描述
closest-side	渐变中心距离容器最近的边作为终止位置
closest-corner	渐变中心距离容器最近的角作为终止位置
farthest-side	渐变中心距离容器最远的边作为终止位置
farthest-corner	默认值。渐变中心距离容器最远的角作为终止位置



我们直接通过例子来理解

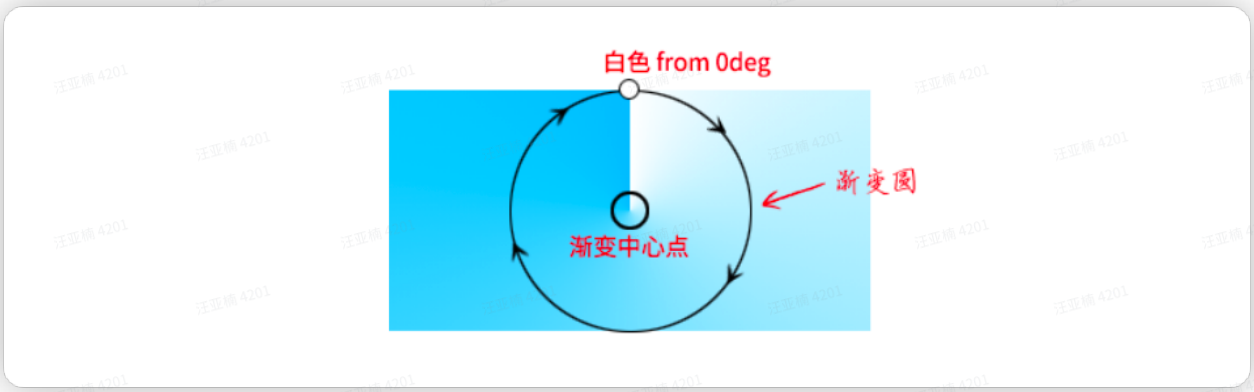
https://codepen.io/dqqbl/pen/GRyxmNR

2.8.3 conic-gradient 锥形渐变

标准语法：

```
CSS
conic-gradient( [ from <angle> ]? [ at <position> ]?, <angular-color-stop-list> )
```

由起始角度，中心位置，角渐变断点组成



我们还是直接来看例子

https://codepen.io/dqqbl/pen/popLPVd

这里有个小技巧，当颜色断点的起始位置小于上一个断点的结束位置时，会以上一个颜色断点的结束位置为起始位置，这样我们就不用自己去计算起始位置

2.8.4 重复渐变

重复渐变仅需要在普通渐变属性前加上 `repeating-`

CSS

```
repeating-linear-gradient(transparent, deepskyblue 40px);
repeating-radial-gradient(transparent, deepskyblue 40px);
repeating-conic-gradient(transparent, deepskyblue 40deg);
```

重复渐变常用于实现条纹效果

https://codepen.io/dqqbl/pen/vYpRZBG

需要注意的是 渐变 属于 <image> 数据类型，该类型是不支持过渡效果的，所以transition属性对渐变无效

2.9 3D变换

2.9.1 基本属性

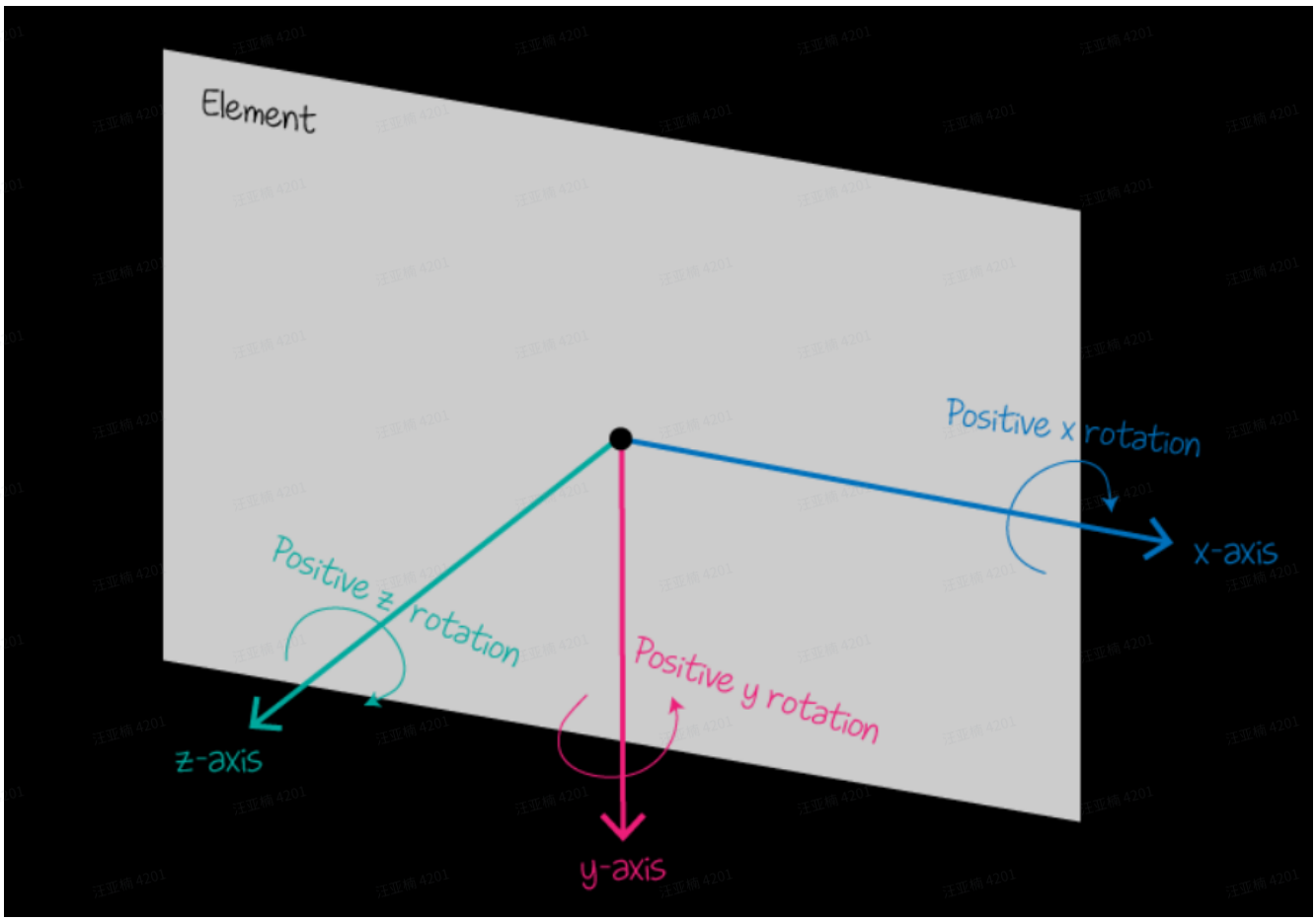
与2d变换类似，3d变换通过 `transform` 的以下函数实现：

- translateZ(0)
- translate3d(0, 0, 0)
- rotateX(0deg)
- rotateY(0deg)
- rotateZ(0deg)
- rotate3d(1, 1, 1, 45deg)
- scaleZ(1)
- scale3d(1, 1, 1)

CSS

```
matrix3d(
 1, 0, 0, 0,
 0, 1, 0, 0,
 0, 0, 1, 0,
 0, 0, 0, 1
)
```

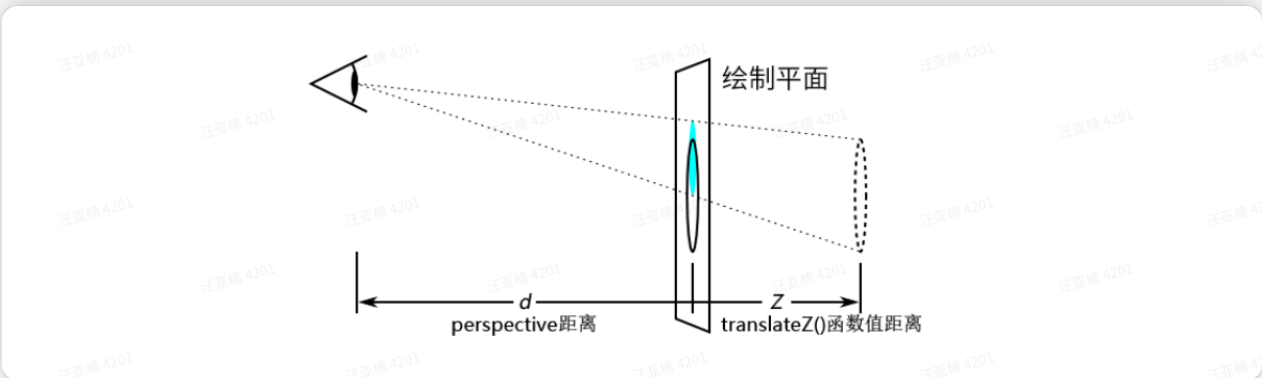
3d变换的重点在于理解两套坐标系，一套坐标系用于平移变换，这套坐标系是相对于元素自身位置的；另一套坐标系用于确定旋转方向，如下图



我们可以通过操作MDN的例子感受一下

2.9.2 perspective / perspective-origin

perspective用于设置透视点到元素坐标原点的距离，perspective-origin属性表示透视点相对3D 变换元素的位置，其值是 <position>，用法与background-position属性等类似，例



2.9.3 其它属性

- transform-style

该属性用于设置元素的子元素是位于 3D 空间中还是平面中，我们还是直接来看下MDN的例子

- backface-visibility

  默认情况下，元素翻转后，背面展示的内容是元素的镜像，但现实世界中显然不是这样，该属性用于控制元素旋转后，元素背面是否展示，取值有visible和hidden；例

  通过该属性，我们可以配合其它元素来实现模拟元素的背面

2.10 css过渡(transition)

`transition`属性可以被指定为一个或多个 CSS 属性的过渡效果，它是transition-duration、transition-delay、transition-property、 transition-timing-function四个属性的缩写

- transition-duration：过渡时间，它的值可以是0， 但是不能是负值
- transition-delay：延迟过渡效果执行的时间， 单位是s(秒)或者ms(毫秒)，其值可以是负值
  - 当值为负值时，部分动画进程会被省略
  - 用于hover浮层，延时出现，避免误触，可提升交互体验
- transition-property：该属性用来设置应用过渡效果的CSS属性，其初始值是all，表示默认所有CSS属性都应用过渡效果，需要注意的是
  - `transition: all .2s` 的 all完全可以省略
  - `display: none/block` 不支持过渡效果，如设置了transition过渡效果的元素应用`display: none`时，过渡效果会被瞬间中断，导致transitionend事件不会被触发；因为display为none的元素会被从文档流中移除，transition无法对从无到有的元素生效，可以用`visibility` 属性替代
- transition-timing-function：用于设置过渡时间函数，来展示过渡效果，主要有三大类
  - 线性运动类型:使用linear表示
  - 三次贝塞尔时间函数类型:ease、ease-in、ease-out、 ease-in-out等关键字和cubic-bezier()函数
  - 步进时间函数类型:step-start等关键字和 steps()函数

该属性的默认值是ease，各曲线效果可以在 这里 查看，一个常见的渐入渐出效果

2.11 css动画(animation/@keyframes)

animation用于指定元素的动画效果，该属性是多个CSS属性的缩写，这些属性包括 animation-name、animation-duration、animation- timing-function、animation-delay、animation-iteration- count、animation-direction、animation-fill-mode和animation-play-state

2.11.1 @keyframes

@keyframes用于定义动画，其语法如下，由动画名和关键帧列表构成：

CSS

```
@keyframes <keyframes-name> {
 <keyframe-block-list>
}

@keyframes fadeIn {
  from { opacity: 0; }
  to   { opacity: 1; }
}
@keyframes fadeIn {
  0%   { opacity: 0; }
  100% { opacity: 1; }
}
```

@keyframes需要注意的点有：

- 起止关键帧可以不设置，默认表现为元素应用动画前的样式
- 如果关键帧对应的CSS样式是一样的，则可以合并在一起书写
- 不同关键帧选择器书写时可以是无序的

CSS

```
@keyframes blink {
  25%, 75% {
    opacity: 1;
  }
  0%, 50%, 100% {
    opacity: 0;
  }
}
```

- 类似于js合并对象，重复定义的关键帧只会对相同的属性进行覆盖
- 关键帧中的样式效果优先级最高，优先级**高于!important**

2.11.2 animation

- animation-name 用于定义动画名，标准语法如下

CSS

```
animation-name = <custom-ident> | <string>
```

`<custom-ident>` 指用户自定义字符串标识符。区分大小写，值不能有任何歧义，不能是CSS属性本身支持的关键字，不能以十进制数字开头

`<custom-ident>` 语法同CSS属性名相似，但它是区分大小写的。可以由以下字符组成：

- 字母（`A` - `Z`，`a` - `z`），
- 十进制数（`0` - `9`），
- 连字符（`-`），
- 下划线（`_`），
- 转义字符（`\`），
- Unicode 编码（格式：转义字符（`\`）后跟1到6位十六进制数）

- animation-delay：可以让动画延时播放，与transition-delay类似，负值可以省略部分动画进程，需要注意的是，如果动画是无限循环的，设置的延时只会生效一次，例
- animation-direction：用于指示动画是否反向播放，它有以下属性值
  - normal，动画执行的方向是0%→100%、0%→100%，每一轮的动画方向都是正常的。
  - reverse，动画执行的方向是100%→0%、100%→0%，每一轮的动画方向都是反向。
  - alternate，动画执行的方向是0%→100%、100%→0%，初始正向执行，且循环时交替正反
  - alternate-reverse，动画执行的方向是100%→0%，0%→100%，初始反向执行，且循环时交替正反
- animation-iteration-count：用于指定动画播放的次数
  - 值可以是小数，小数则播放到中途停止
  - 值为infinite关键字，则动画无限循环
- animation-fill-mode：该属性的字面意思是“动画填充模式”， 主要用来定义动画在执行时间之外应用的值；通俗的讲就是定义动画开始前和结束后元素的样式表现
  - none：默认值，表示动画执行完后恢复元素执行动画前的样式
  - forwards：forwards是“前进”的意思，表示动画执行结束之后，保持终止帧的样式
  - backwards：表示动画执行前，元素保持起始帧的样式
  - both：表示 forwards + backwards
  - 需要注意的是，若没有指定起止帧样式，则起止帧样式使用的是元素应用动画前的样式

https://codepen.io/dqqbl/pen/BaJrYdK

- animation-play-state：用于动画的启动与暂停
  - running 播放
  - paused 暂停
  - 需要用js添加删除

一些使用技巧：

- animation支持同时应用多个动画规则，因此对于不同效果的动画建议进行拆分，方便复用，如一个元素淡出和右侧划入同时进行的动画效果，可以拆分开来

CSS

```
animation: fadeInSlideInRight .2s;
/* 拆分成 */
animation: fadeIn .2s, slideInRight .2s;
```

- 对于冲突的动画效果可以通过嵌套解决，比如通过transform实现的居中定位，又希望在动画里使用 `tranform: scale(3)` 放大元素
- 重新执行动画：

https://codepen.io/dqqbl/pen/oNpqqNN